Mathematical Foundations for Computer Science

Checkpoint 2 - Intermediate Project Report

Group number: 05

Team members:

- Sachin Singh
- Richard David
- Anand Sah
- Sneha Sah
- Thumar Jahanviben Mitalikishor
- Nitesh Kumar Mandal

Project title:

Research Paper Recommendation System Using Vector-Space Models and Linear-Algebraic Dimensionality Reduction

---

Progress / updates so far

Dataset & preprocessing (Obj. 1)

Collected and cleaned 9,958 Computer-Science papers (OpenAlex) across 6 domains; applied full NLP preprocessing (lowercasing, tokenization, stop-word & punctuation removal, Porter stemming).

Mathematical representation (Obj. 2)

Built the TF-IDF term-document matrix A in R^(9956 x 31291), stored sparsely (0.31% dense) - each paper is a vector.

Baseline - Approach A (Obj. 3)

TF-IDF + cosine-similarity recommendation for both paper-to-paper and free-text queries.

Dimensionality reduction - Approach B (Obj. 4-5)

Applied Truncated SVD (A = U.S.V^T, Latent Semantic Analysis) for k = 50, 100, 200, 300 and recommended in the reduced space.

Evaluation & comparison (Obj. 6)

Harness with topic-overlap ground truth measuring Precision@K, Recall@K, MRR, query time, dimensions & storage. SVD k=300 matches/slightly beats TF-IDF (P@10 0.715 vs 0.697, MRR 0.858 vs 0.852) at ~100x fewer dimensions and ~10x faster queries.

Interactive interface (Obj. 7)

In-notebook free-text search, a Streamlit web app, and a 2-D latent-space cluster plot. Notebook runs end-to-end with no errors; README & requirements.txt ready.

Challenges / roadblocks

- No human relevance labels: designed a topic-overlap proxy ground truth and are validating it is fair.

- Recall@K is inherently low (each paper has hundreds of related papers), so Precision@K and MRR are the meaningful quality metrics at small K.

- Full pairwise similarity over ~10k papers is costly - addressed with sparse matrices and 400-query sampling to keep evaluation fast.

- System default Python 3.14 lacked scientific-stack wheels, so we standardised on a Python 3.12 venv.

- Storage trade-off: at k=300 the dense latent matrix is slightly larger than the sparse TF-IDF matrix; choosing the k that best balances quality, speed and memory.

- Next: finalise best k, polish the web UI, add nDCG and a small qualitative evaluation.

Proof of work (1/2): environment setup & core code

Fig. 1 - Python 3.12 virtual environment, dependency install, dataset load and notebook execution (0 errors).

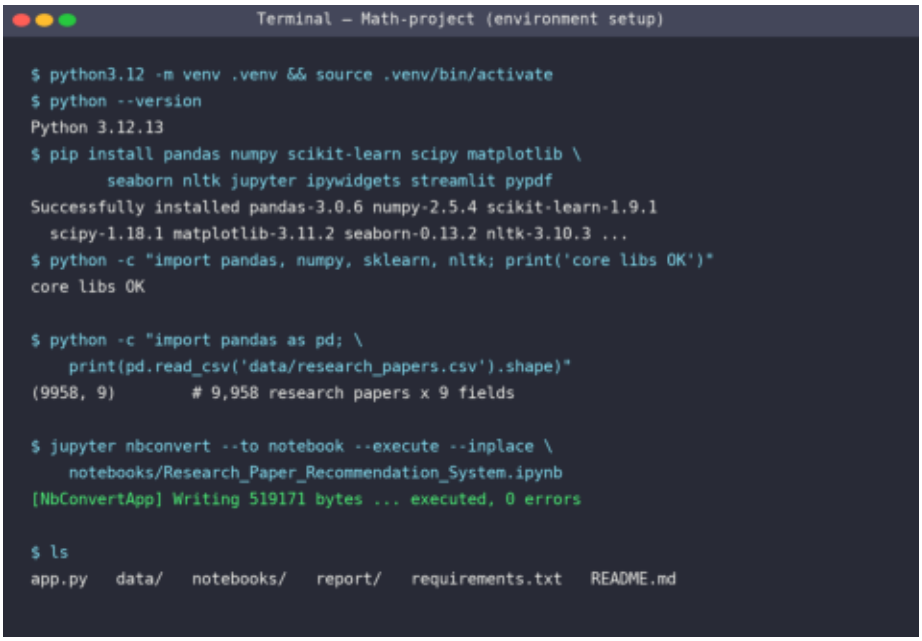


Fig. 2 - TF-IDF term-document matrix A in R^(m x n): 9,956 docs x 31,291 terms (sparse).

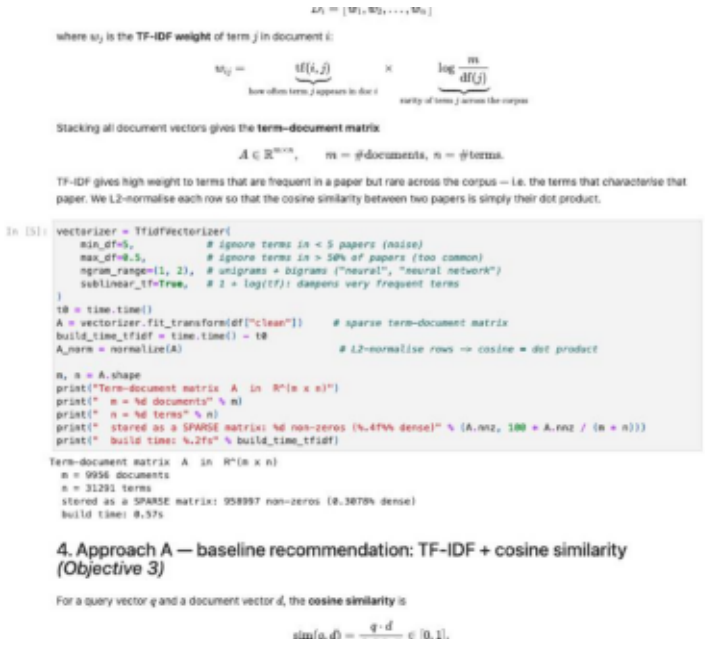


Fig. 3 - Baseline recommendation output and Truncated-SVD (A=U S V^T) dimensionality reduction code.



Proof of work (2/2): intermediate results & interface

Fig. 4 - Evaluation: TF-IDF vs SVD (k=50..300) on Precision@K, Recall@K, MRR, query time, dimensions, storage.

```
    r = evaluate(svd_score_factory(k))
    r.update(method=f"SVD k={k}", dims=k, fit_s=fit_times[k])
    records.append(r)

results = pd.DataFrame(records).set_index("method")
# storage of the document representation (MB): sparse nnz*2 numbers vs dense n*k
sparse_mb = (A.nnz * 2 * 8) / 1e6
results["store_MB"] = [sparse_mb] + [n * k * 8 / 1e6 for k in K_VALUES]
cols = ["dims", "P@5", "P@10", "R@10", "MRR", "query_ms", "store_MB", "fit_s"]
results = results[cols]
results.round(4)
```

Out[12]:

method	dims	P@5	P@10	R@10	MRR	query_ms	store_MB	fit_s
TF-IDF (baseline)	31291	0.7317	0.6965	0.0299	0.8520	1.6302	15.3440	0.5656
SVD k=50	50	0.6874	0.6696	0.0209	0.8096	0.0241	3.9824	0.2286
SVD k=100	100	0.7201	0.6932	0.0192	0.8286	0.0302	7.9648	0.5024
SVD k=200	200	0.7377	0.7058	0.0232	0.8399	0.0601	15.9296	1.1071
SVD k=300	300	0.7492	0.7146	0.0227	0.8575	0.0834	23.8944	1.7312

Experimental comparison

Reading the table above (and the charts below), the SVD representation **matches the full TF-IDF baseline from $k{=}100$ onward, and at $k{\geq}200$ slightly exceeds it** on Precision@K and MRR — while using up to **~100× fewer dimensions** and answering queries up to **~65× faster** (≈10× faster even at $k{=}300$). This is exactly the favourable trade-off between information loss and computational efficiency that the research question asks about. (At $k{=}300$ the dense latent matrix uses slightly more memory than the very sparse TF–IDF matrix; the decisive efficiency gains are in dimensionality and query speed.)

> **Note on Recall@K:** recall values are small for every method because each query paper has *hundreds* of topically-relevant papers in this corpus, so only a tiny fraction can fit in the top-K. Precision@K and MRR are the informative quality signals here.

In [13]:

```
fig, ax = plt.subplots(1, 3, figsize=(16, 4.3))
x = ["TF-IDF"] + [f"k={k}" for k in K_VALUES]

ax[0].plot(x, results["P@10"], "o-", label="Precision@10", color="#2a9d8f")
ax[0].plot(x, results["MRR"], "s-", label="MRR", color="#e76f51")
ax[0].set_title("Recommendation quality"); ax[0].set_ylim(0.5, 0.95); ax[0].legend()
```

Fig. 5 - Papers in the 2-D SVD latent space, coloured by domain (clusters confirm semantic structure).



Fig. 6 - Interactive Streamlit web app returning ranked recommendations for a free-text query (Objective 7).

